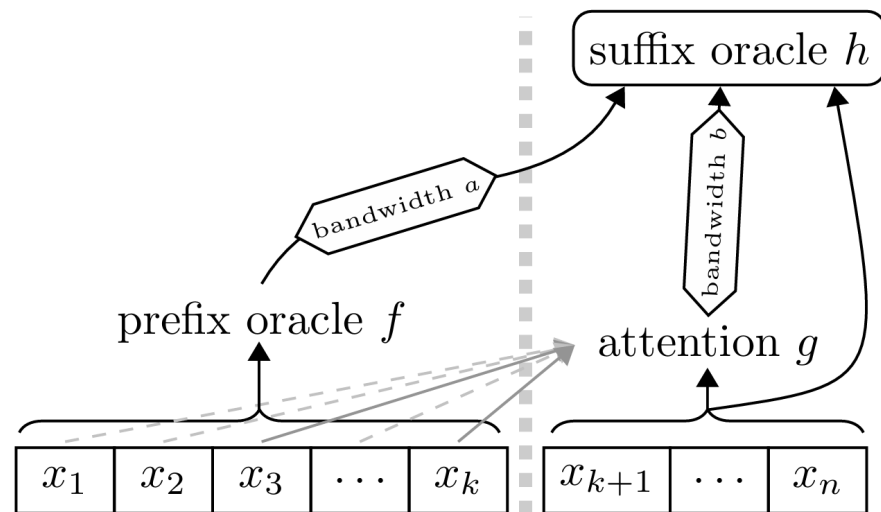
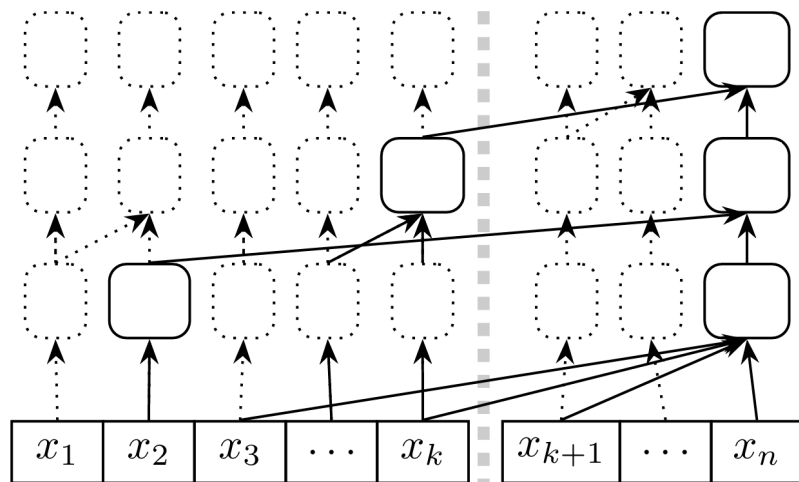
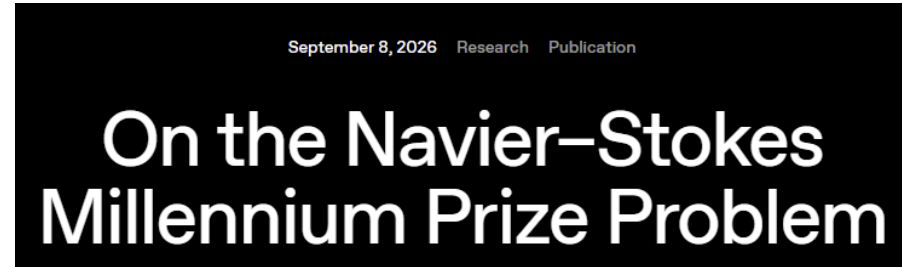


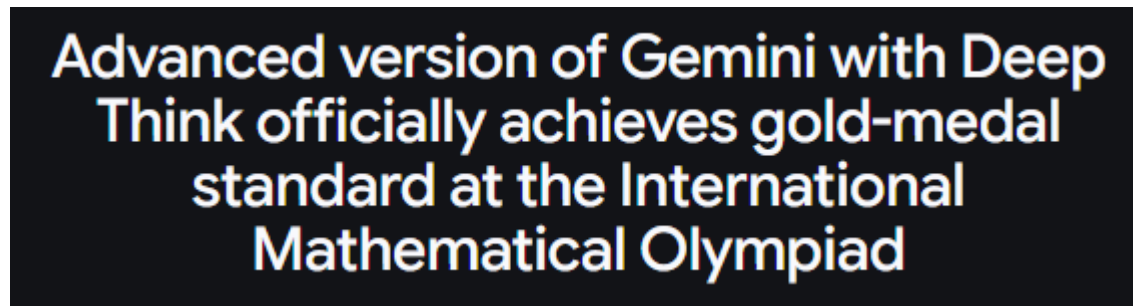


BAPO: A Communication Complexity Model of Transformer Capabilities

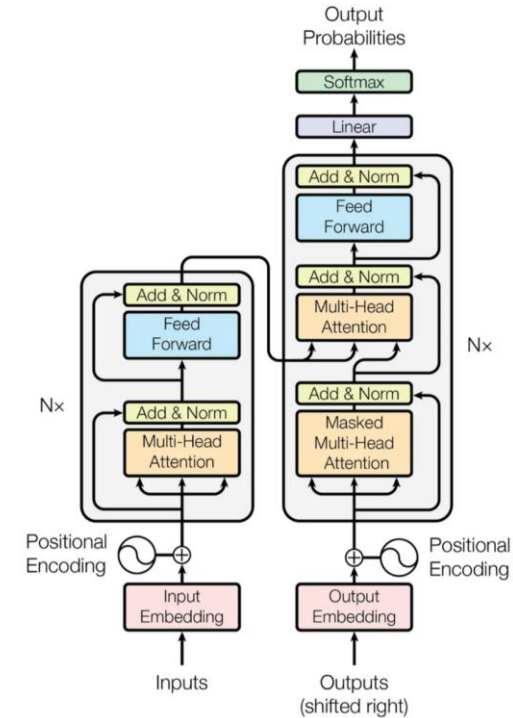
Transformer-based LLMs have had incredible success.



[OpenAI 2026]



[DeepMind 2025]



[Vaswani et al. 2017]

*But what are the theoretical capabilities
of the underlying architecture?*

Overview of transformer theory

Circuit complexity

Transformer expressivity upper bound: (DLOGTIME-uniform) **TC⁰** [Chiang 2024]
[Merrill and Sabharwal 2023] [Strobl 2023]

Formal logic / RASP [Weiss et al. 2021]

Transformers equivalent to **C-RASP** [Yang et al. 2024] [Yang et al. 2026]

Chain of thought

Turing-complete [Pérez et al. 2021] [Merrill and Sabharwal 2024]

Lots of other work!

sparsity [Sanford et al. 2023] [Hahn and Rofin 2024] [Edelman et al. 2022]

MPC/MapReduce [Sanford et al. 2024]

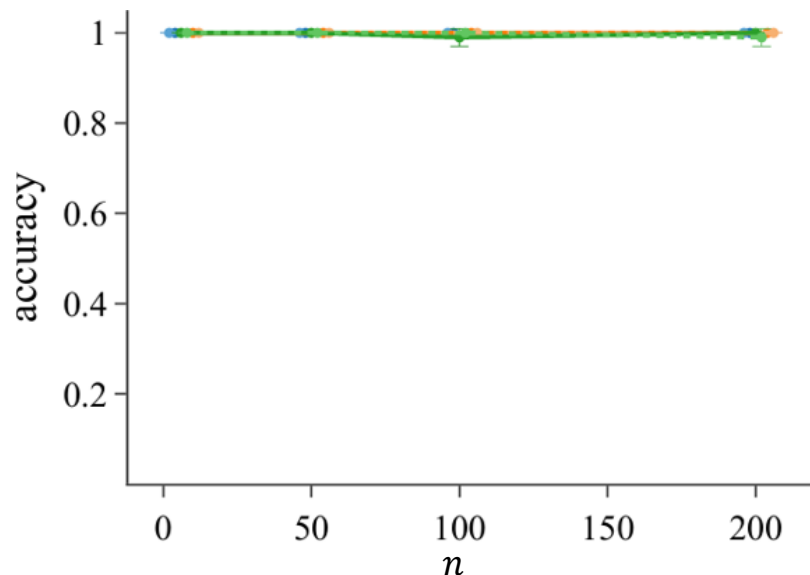
automata shortcuts [Liu et al. 2023]

limits of attention [Hahn 2020]

survey: [Strobl et al. 2024]

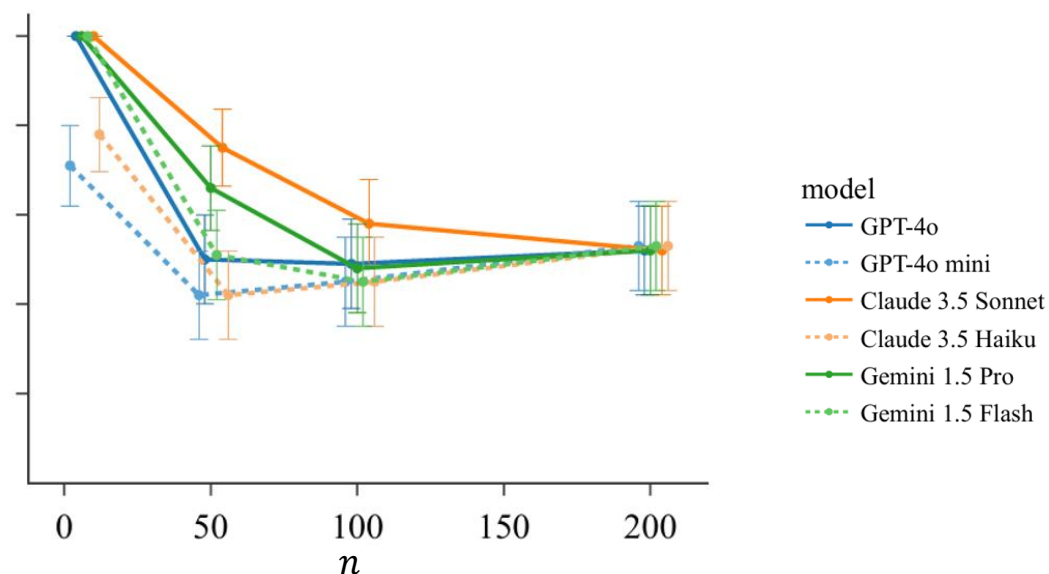
Existing theory doesn't always align with practice

Index: $p(x_{1\dots n}, i) = x_i$



Hard* under C-RASP

Majority: $p(x_{1\dots n}) = \mathbf{1}[\sum_{i=1}^n x_i > n/2]$



Easy under TC^0

opposite of LLM behavior

Goal: theoretical model that captures hard tasks for LLMs

Talk outline

1. **Background**
2. The BAPO model [1]
3. BAPO results [1]
4. BAPOs with chain of thought [2]



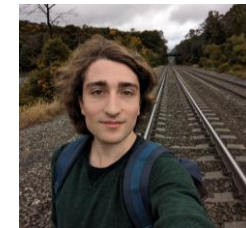
Tobias Schnabel
MSR



Jennifer Neville
MSR



Adith Swaminathan
Netflix



Awni Altabaa
Yale

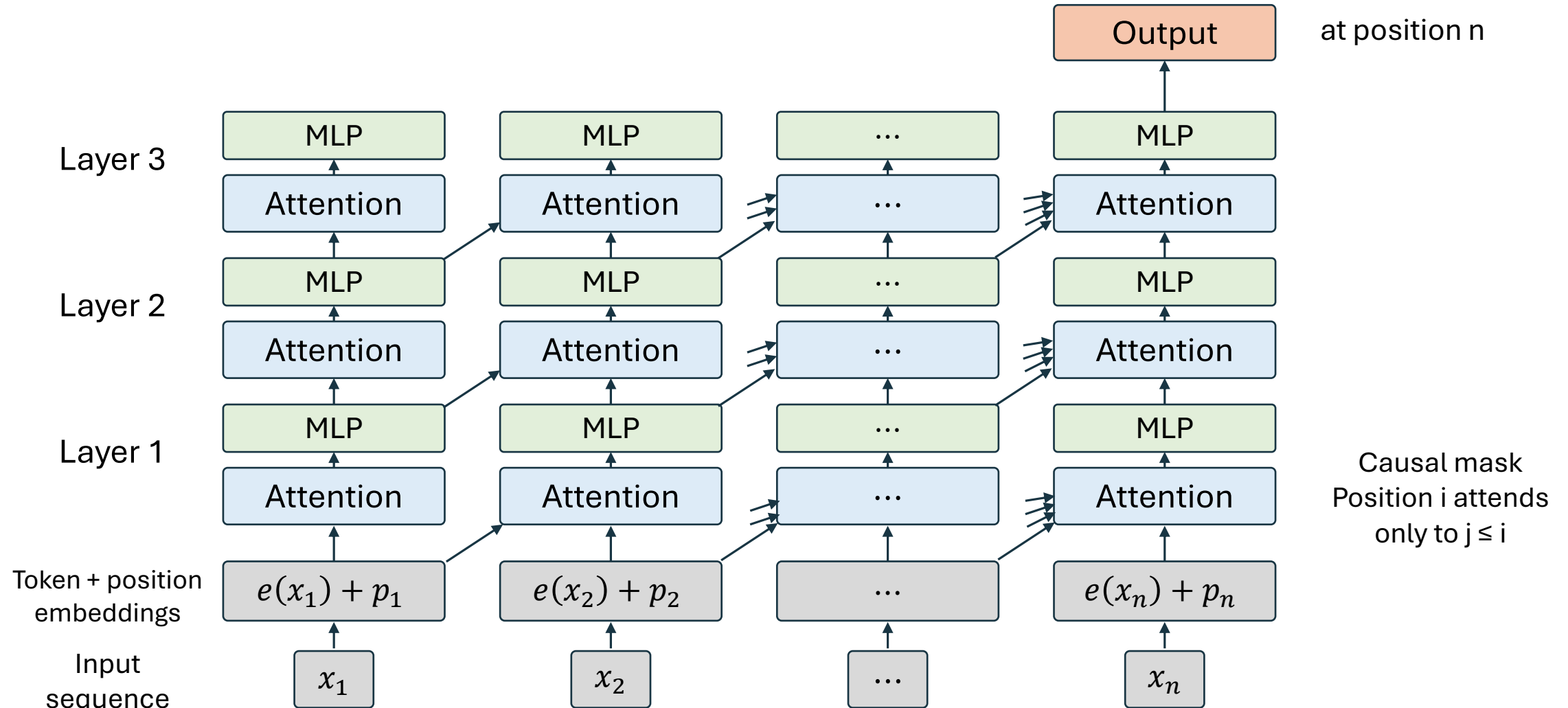
[1] Schnabel, Tomlinson, Swaminathan, Neville. Lost in transmission: When and why LLMs fail to reason globally. *NeurIPS* 2025.

[2] Tomlinson, Schnabel, Swaminathan, Neville. Reasoning about reasoning: BAPO bounds on chain-of-thought token complexity in LLMs. *ICML* 2026.

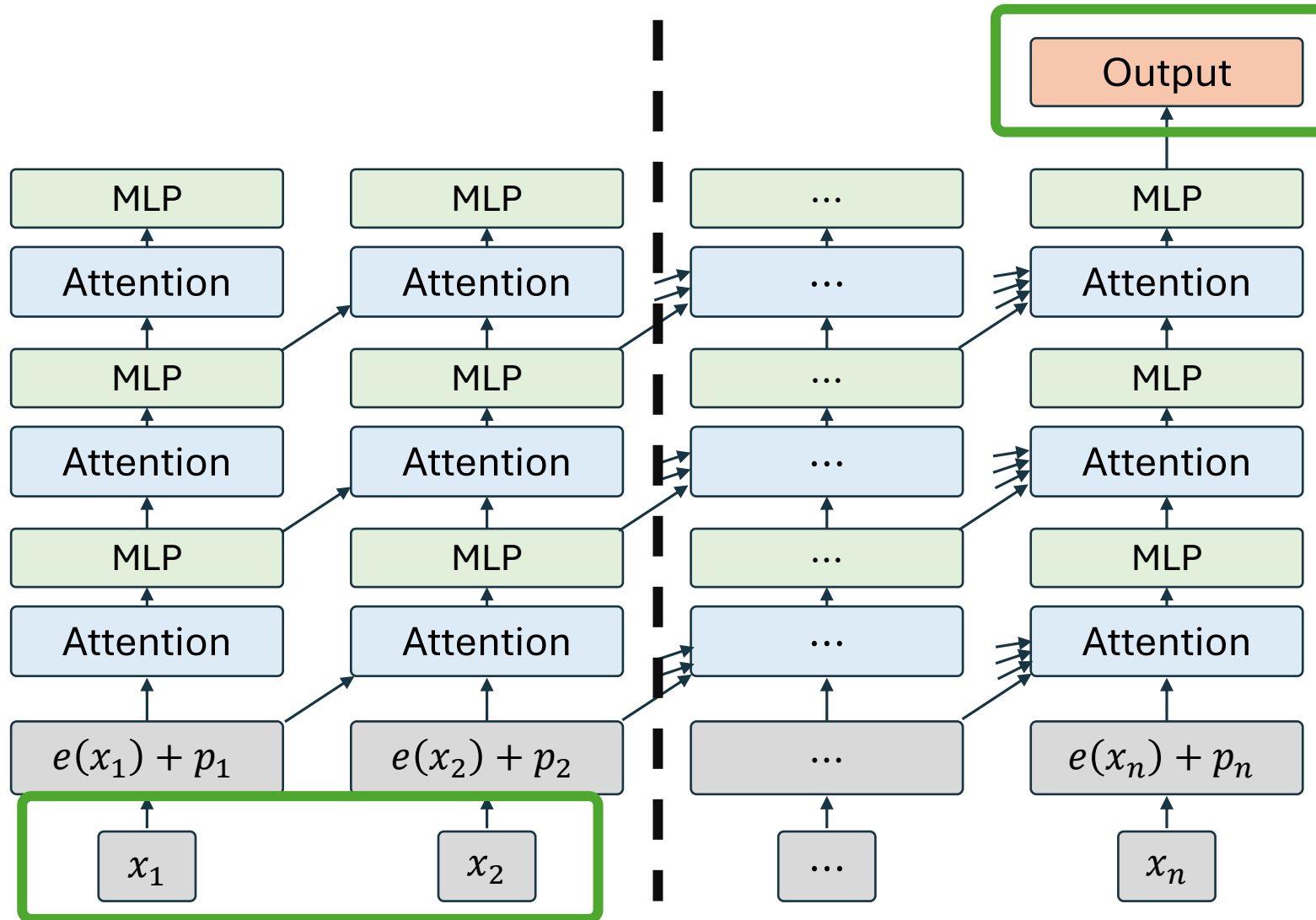
[3] Altabaa, Schnabel, Tomlinson. Putting transformer theories to the test. *Working paper*, 2026.

Transformer primer (causal / “decoder-only”)

[Vaswani et al. 2017]



Information flow in causal transformers



Information could be:

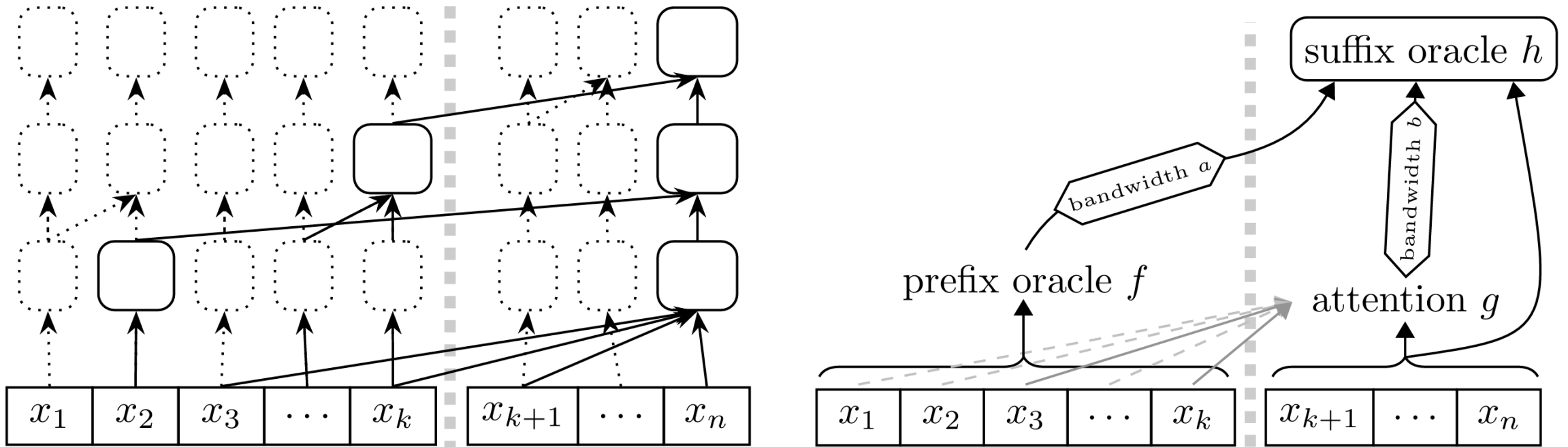
- Token values
- Processed / contextual

Because of causal attention:
- Processed / contextual info must be useful *regardless of the suffix*

Hypothesis:

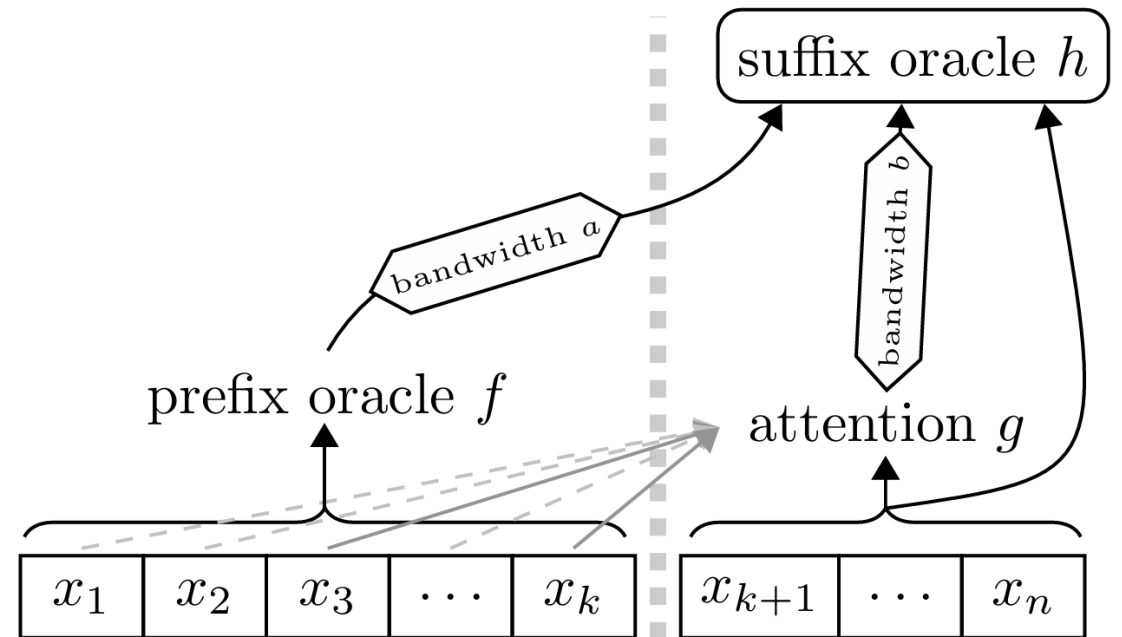
Accurate communication of tokens / processed values is a bottleneck in transformers.

Our communication-inspired model: the *bounded attention prefix oracle* (BAPO)



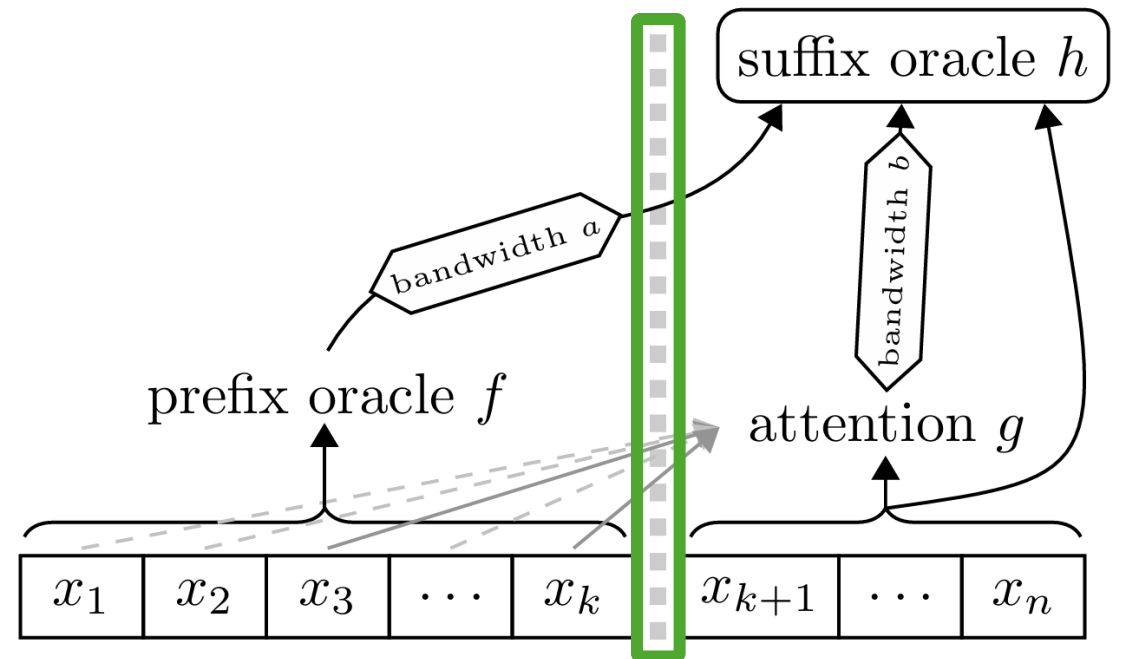
Models bottlenecks in causal information flow through
(a) compressive preprocessing and (b) limited attention

BAPO intuition



BAPO intuition

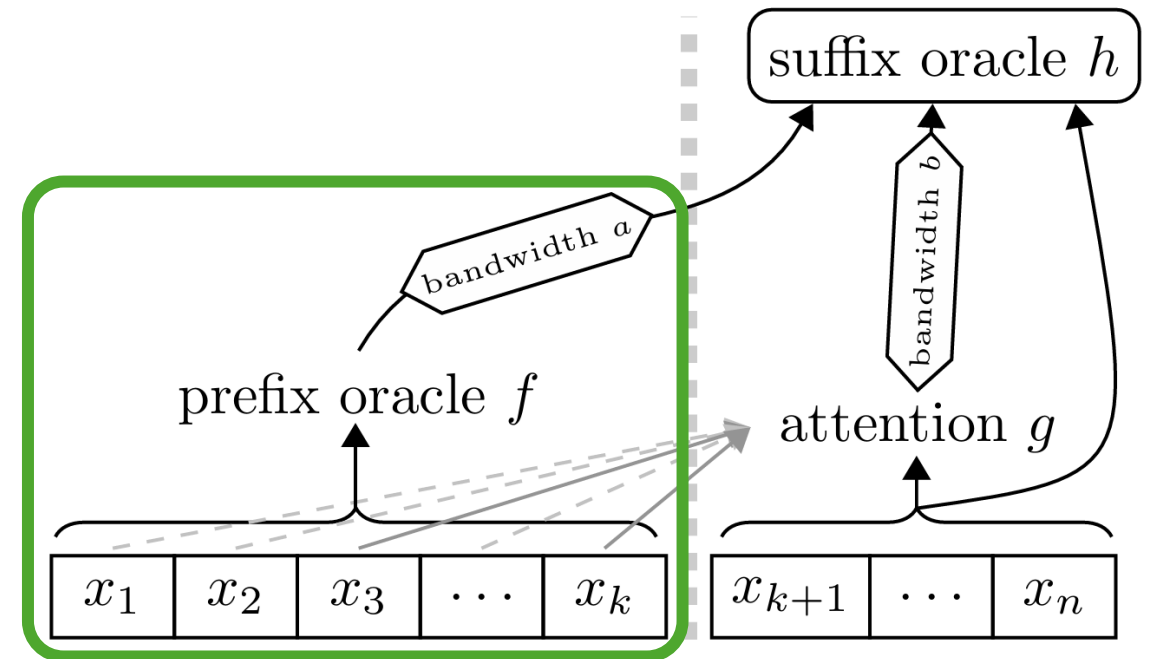
Input split arbitrarily into prefix and suffix



BAPO intuition

Input split arbitrarily into prefix and suffix

f summarizes prefix in a bits (useful *regardless of suffix*)

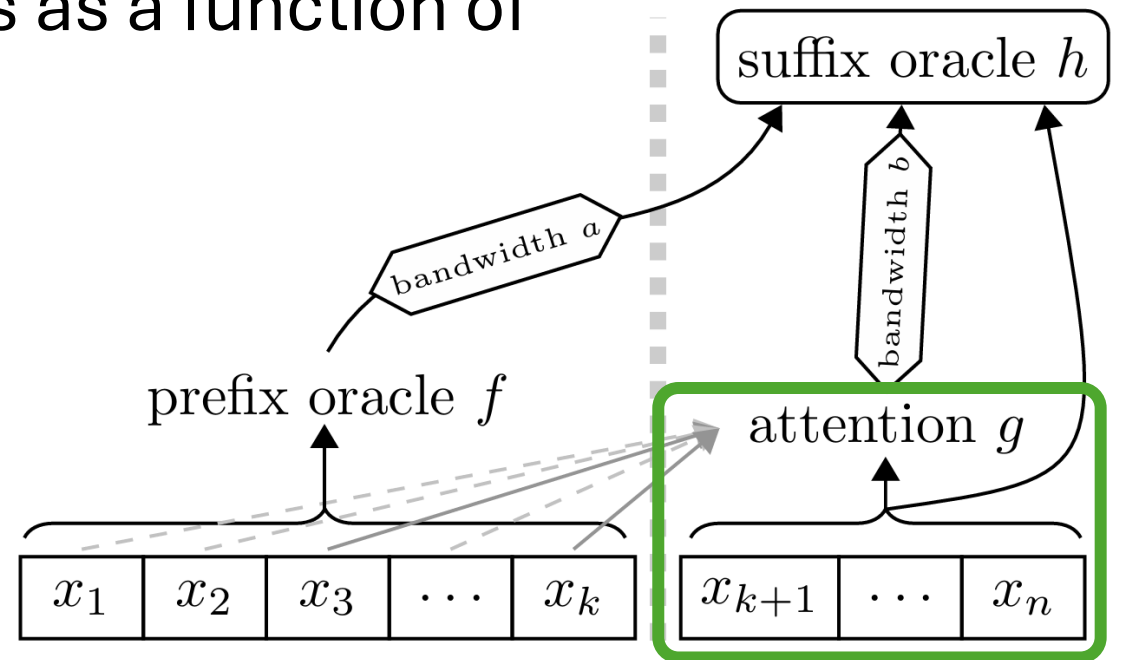


BAPO intuition

Input split arbitrarily into prefix and suffix

f summarizes prefix in a bits (useful *regardless of suffix*)

g independently selects prefix tokens as a function of suffix



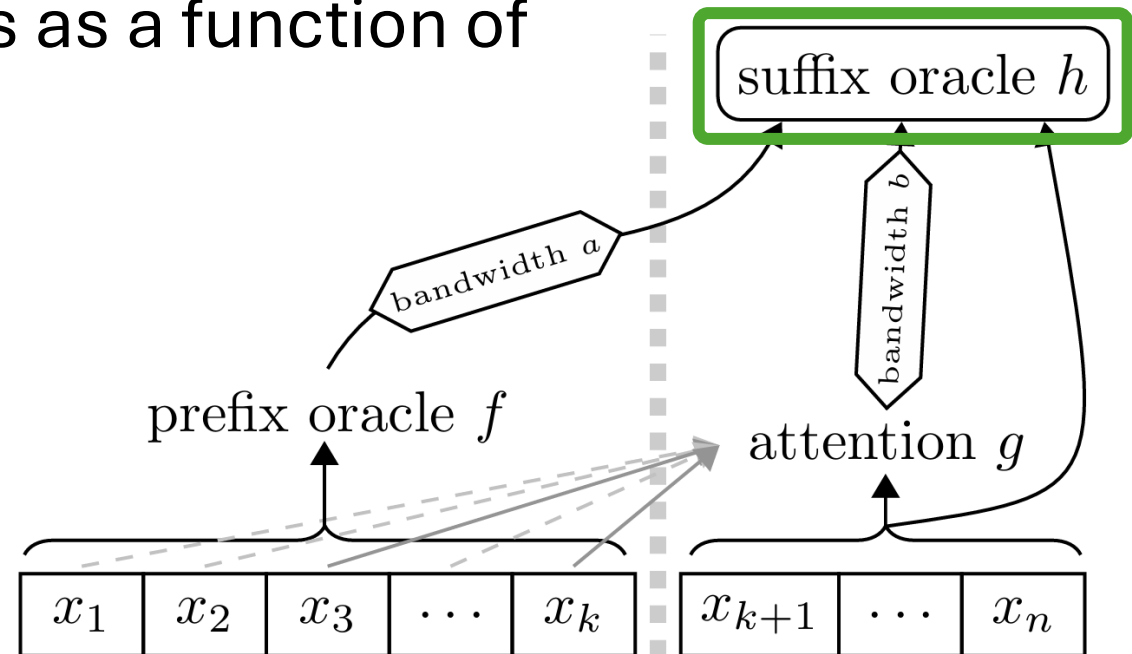
BAPO intuition

Input split arbitrarily into prefix and suffix

f summarizes prefix in a bits (useful *regardless of suffix*)

g independently selects prefix tokens as a function of suffix

h solves a task given:



BAPO intuition

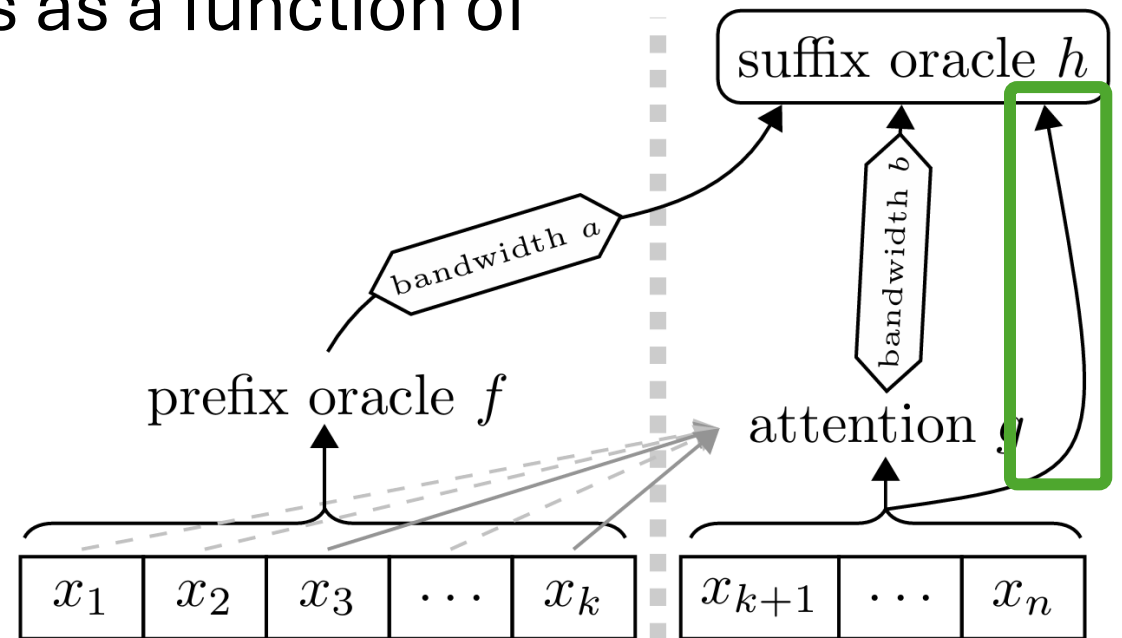
Input split arbitrarily into prefix and suffix

f summarizes prefix in a bits (useful *regardless of suffix*)

g independently selects prefix tokens as a function of suffix

h solves a task given:

- suffix (and split index)



BAPO intuition

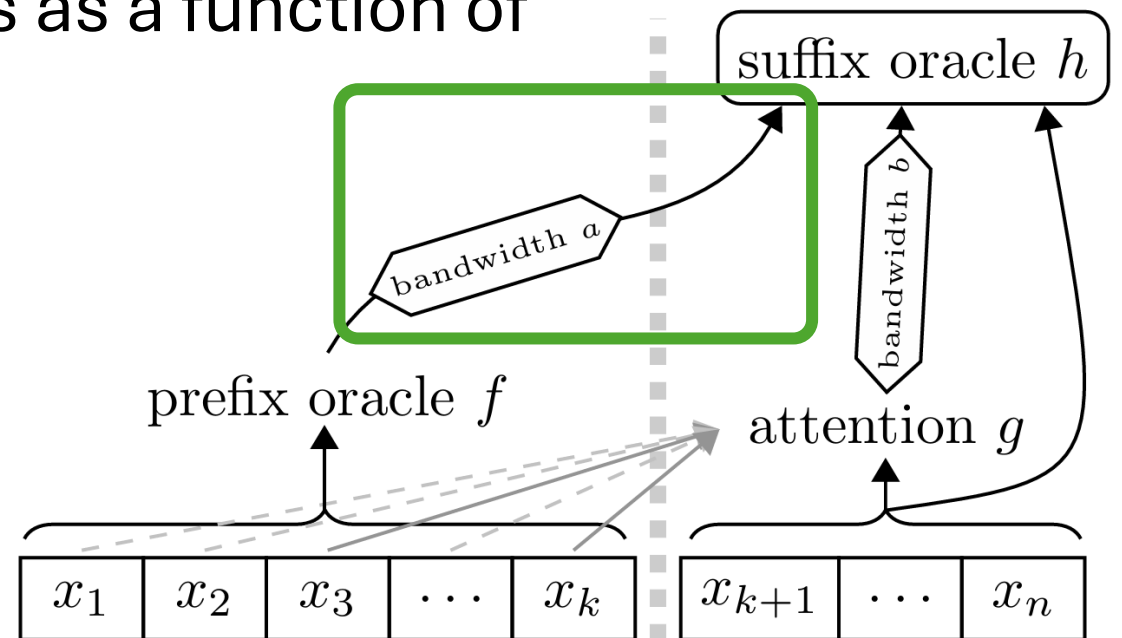
Input split arbitrarily into prefix and suffix

f summarizes prefix in a bits (useful *regardless of suffix*)

g independently selects prefix tokens as a function of suffix

h solves a task given:

- suffix (and split index)
- a bits from f



BAPO intuition

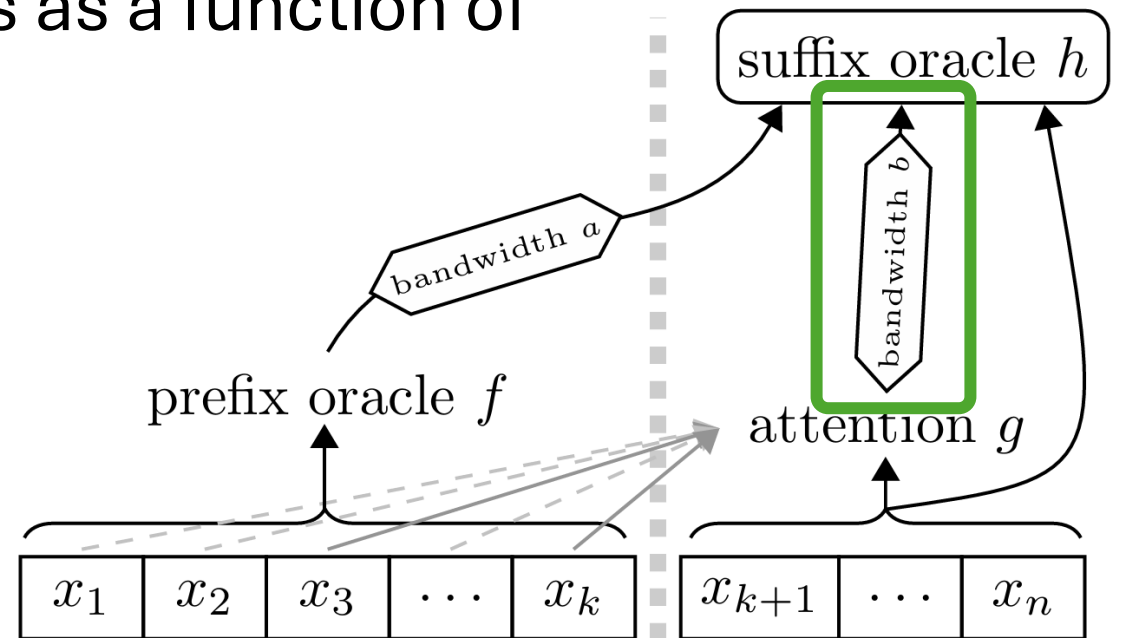
Input split arbitrarily into prefix and suffix

f summarizes prefix in a bits (useful *regardless of suffix*)

g independently selects prefix tokens as a function of suffix

h solves a task given:

- suffix (and split index)
- a bits from f
- b prefix tokens selected by g



BAPO intuition

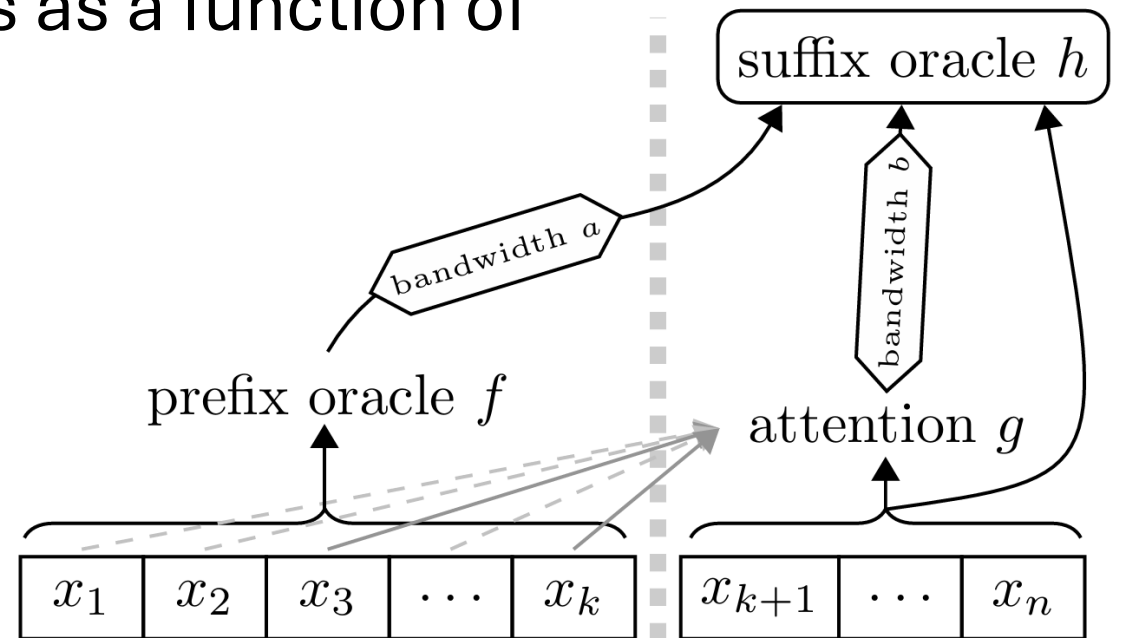
Input split arbitrarily into prefix and suffix

f summarizes prefix in a bits (useful *regardless of suffix*)

g independently selects prefix tokens as a function of suffix

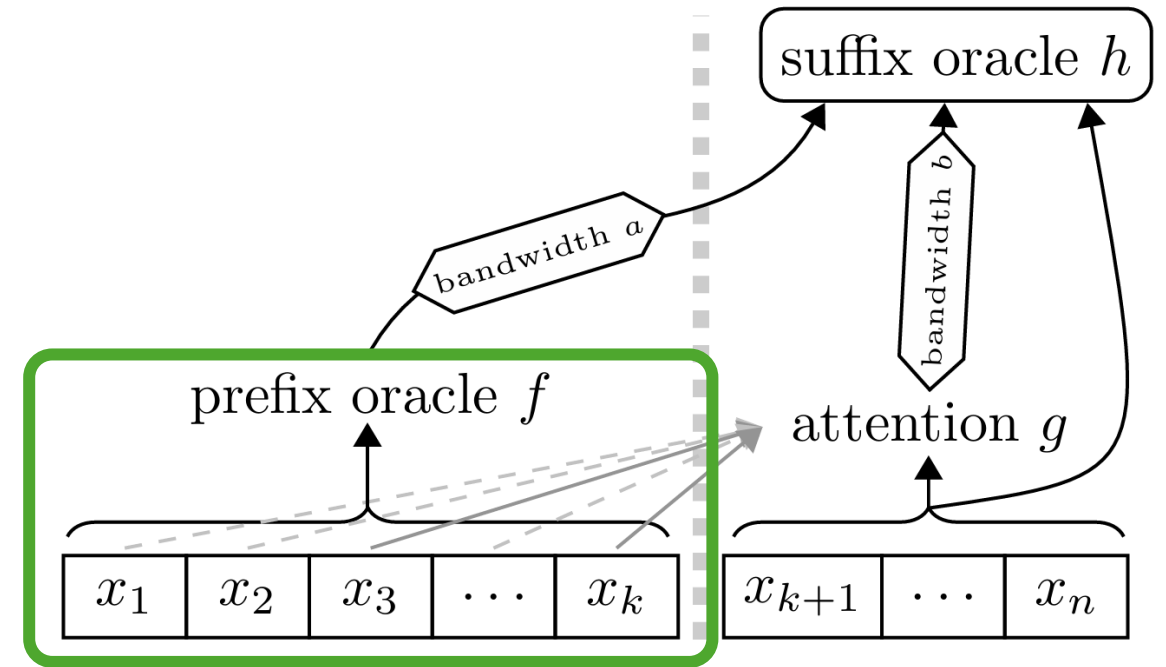
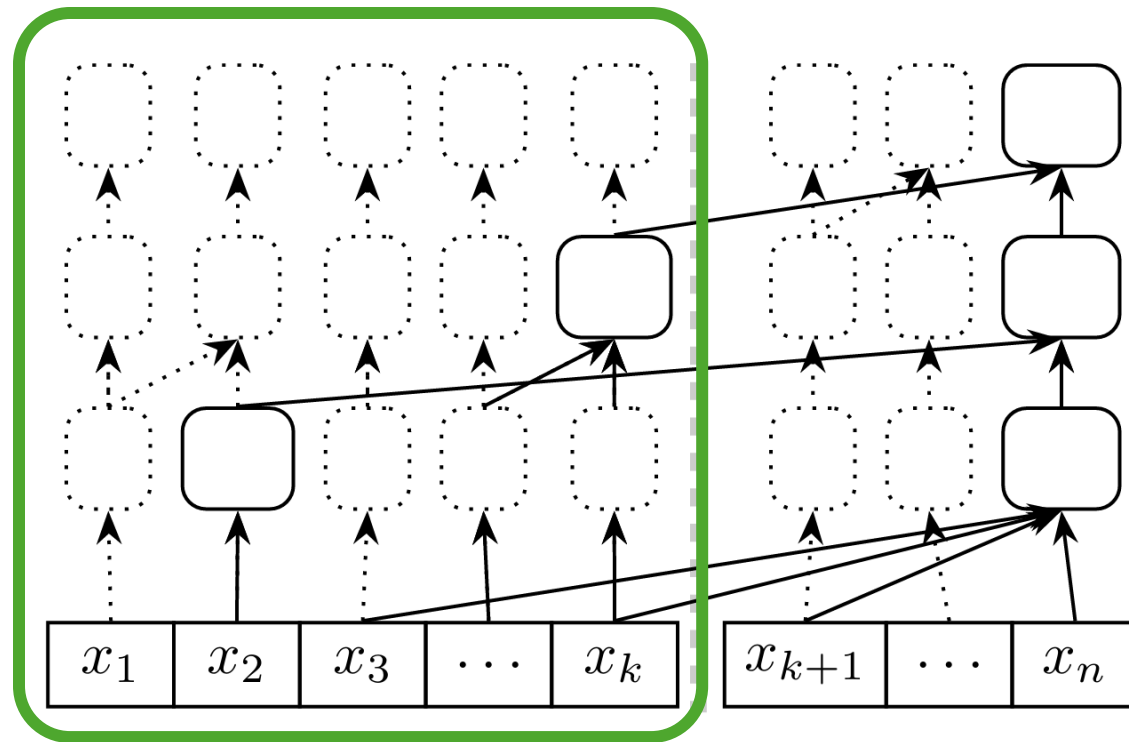
h solves a task given:

- suffix (and split index)
- a bits from f
- b prefix tokens selected by g



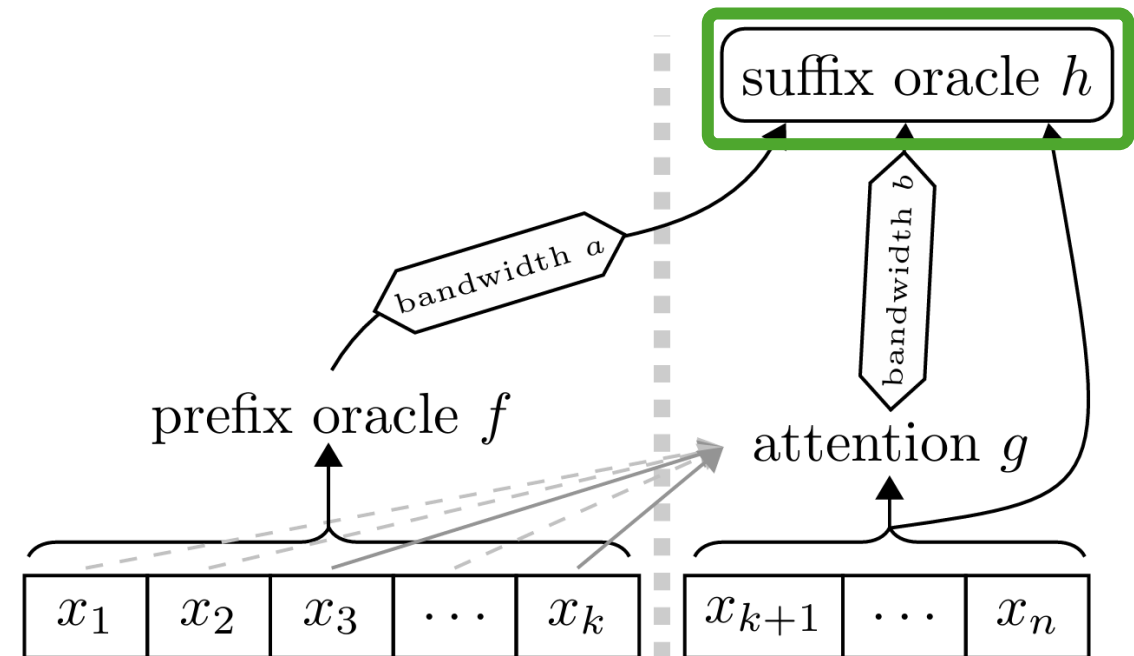
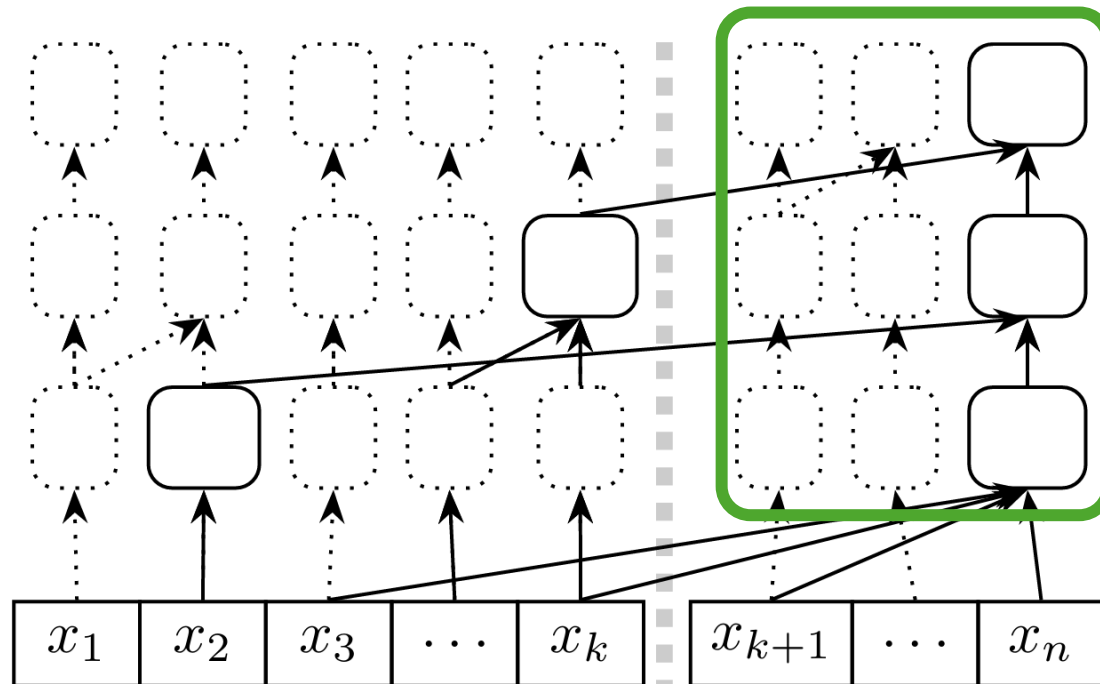
BAPO - transformer analogy

(not formal correspondence)



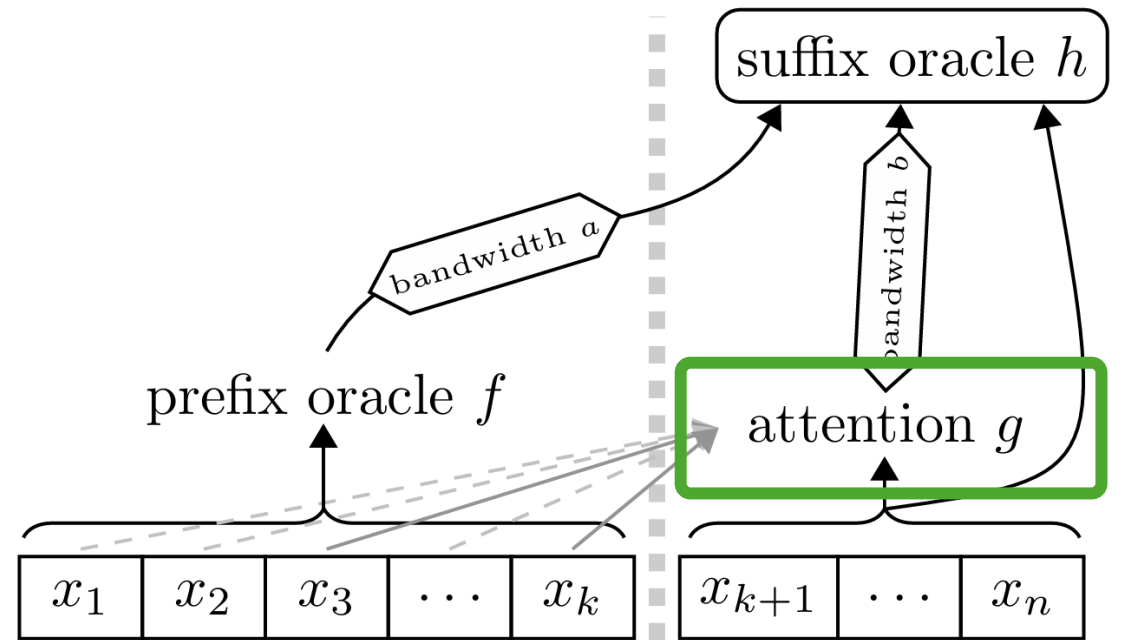
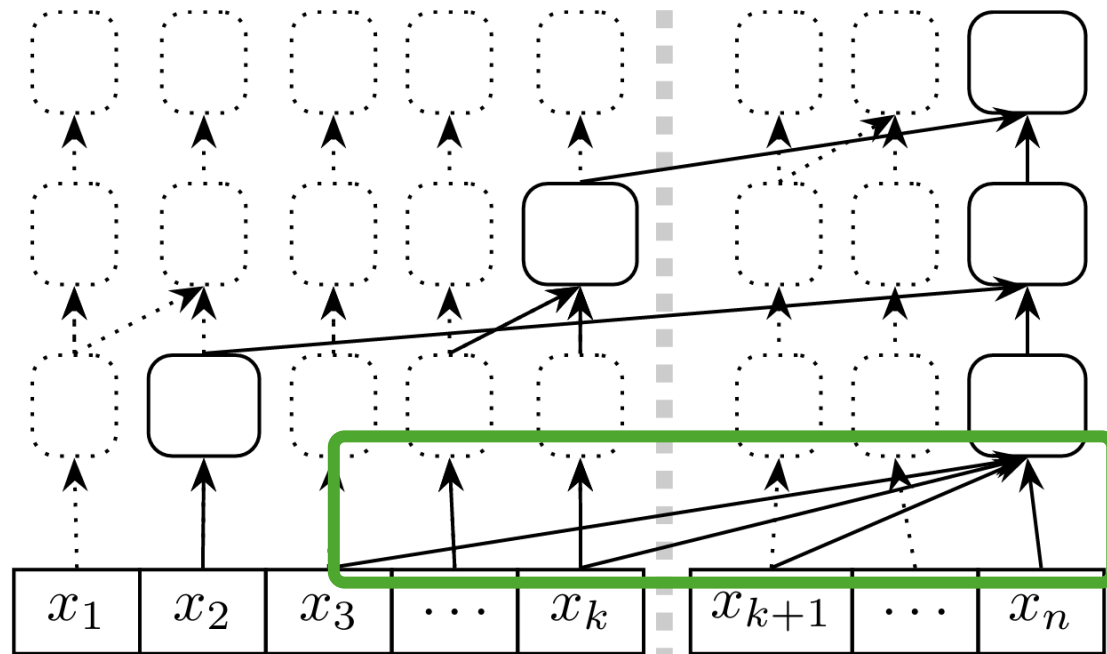
BAPO - transformer analogy

(not formal correspondence)



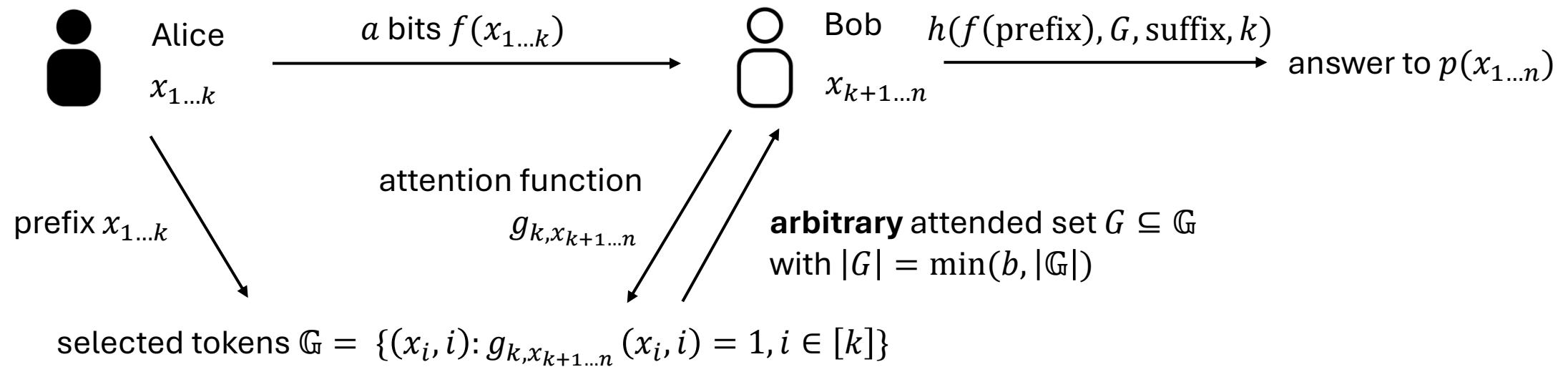
BAPO - transformer analogy

(not formal correspondence)



BAPOs: communication view, more formally

On input $x_{1\dots n} \in \Sigma^*$, input is **arbitrarily** split at index k :



Definition. An (a, b) -BAPO f, g, h solves p if the above protocol works for all splits $k < n$ and all attended sets G .

BAPO example: Equality

Input: $x_{1\dots n} \# y_{1\dots n}$

Output: $p(x \# y) = \mathbf{1}[x = y]$

One-way deterministic communication complexity n (pigeonhole)

(1, 1)-BAPO for Equality:

$f(x_{1\dots n} \# y_{1\dots \ell}) = \mathbf{1}[x_{1\dots \ell} = y_{1\dots \ell}]$

(check no $x_i \neq y_i$ in prefix)

$g(x_i, i) = \mathbf{1}[x_i \neq y_i]$

(attend to cross-split $x_i \neq y_i$)

h checks suffix, $f() = 1$, and $G = \emptyset$

(verify no $x_i \neq y_i$)

Cases for split index:

01**0**10 # 01**1**1 | 0

f catches

01**0**10 # 0 | 1**1**10

g catches

0 | 1**0**10 # 01**1**10

h catches

Talk outline

1. Background
2. The BAPO model [1]
- 3. BAPO results [1]**
4. BAPOs with chain of thought [2]

[1] Schnabel, Tomlinson, Swaminathan, Neville. Lost in transmission: When and why LLMs fail to reason globally. *NeurIPS* 2025.

[2] Tomlinson, Schnabel, Swaminathan, Neville. Reasoning about reasoning: BAPO bounds on chain-of-thought token complexity in LLMs. *ICML* 2026.

[3] Altabaa, Schnabel, Tomlinson. Putting transformer theories to the test. *Working paper*, 2026.

BAPO-easy tasks

Definition. A problem p is *BAPO-easy* if there are constants a, b such that for any n , there is an (a, b) -BAPO that solves p on inputs of size n .

Note: any problem solved by $(0, n)$ -BAPO and $(n \lceil \log_2 \Sigma \rceil, 0)$ -BAPO

BAPO-easy:

- Equality: $p(x\#y) = \mathbf{1}[x = y]$
- Index: $p(x\#i) = x_i$
- k -sparse functions
- regular language recognition

Bandwidths:

- $(1, 1)$
- $(0, 1)$
- $(0, k)$
- $(\lceil \log_2 |Q| \rceil, 0)$

A first BAPO-hard task: Majority

$(O(\log n), 0)$ -BAPO for Majority:

$$f(x_{1\dots k}) = \sum_i x_i$$

h counts 1s in suffix, adds $f(x_{1\dots k})$, checks majority

Can we do better? *No!*

Theorem. No $(o(\log n), O(n^{1-\epsilon}))$ -BAPO solves Majority for any $\epsilon > 0$.

[Schnabel et al. 2025]

BAPO-hardness proof strategy: fooling set

1. Suppose (a, b) -BAPO f, g, h solves p
2. Construct $> 2^a$ prefix-suffix pairs x_i, y_i such that:
 - a) For any suffix, all prefixes share a possible attended set G
 - b) Given a collision $f(x_i) = f(x_j)$ (exists by pigeonhole), there is a suffix y_ℓ such that $p(x_i y_\ell) \neq p(x_j y_\ell)$

If we can do this, we can fool the BAPO:

given $x_i \mid y_\ell$ and $x_j \mid y_\ell$,

h sees the same suffix y_ℓ , same attended set G , same output of f

Other BAPO-hard problems

Reachability: $p(E, s, t)$ = do the edges in E have an s - t path?

Match3 _{n} : $p(x_1 \dots x_n)$ = are there i, j s.t. $x_i + x_j = x_n$? (~ 2 SUM)

[Sanford et al. 2023]

Theorem. Reachability and Match3 are BAPO-hard.

[Schnabel et al. 2025]

Dyck- k : $p(s)$ = is s a balanced string over k parenthesis types?

BinaryAddition: $p(x, y, z) = \mathbf{1}[x + y = z]$ (binary representation)

Theorem. Dyck- k and BinaryAddition are BAPO-hard.

[Altabaa et al. 2026, forthcoming]

Instantiating the proof technique: Reachability

Goal. Construct $> 2^a$ prefix-suffix pairs x_i, y_i such that:

- a) For any suffix, all prefixes share a possible attended set G
- b) Given a collision $f(x_i) = f(x_j)$ (exists by pigeonhole), there is a suffix y_ℓ such that $p(x_i y_\ell) \neq p(x_j y_\ell)$

Prefixes: edge lists for ℓ length- $\ell^2 b$ paths from s_i to t_j

Suffixes: all pairs s_i, t_j

To fix G :

For each suffix s_i, t_j :

Find b edges that g selects

Add those to a *shared prefix* P

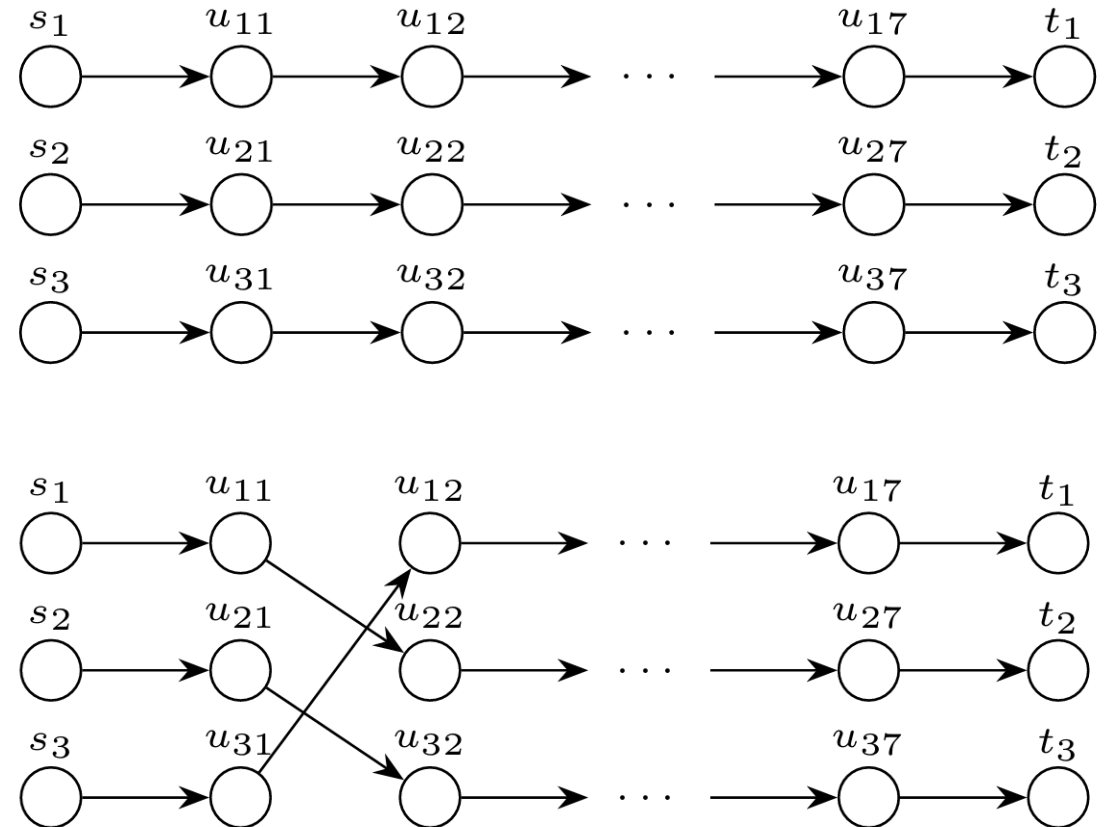
Always have some $G \subseteq P$

P has $\ell^2 b$ edges

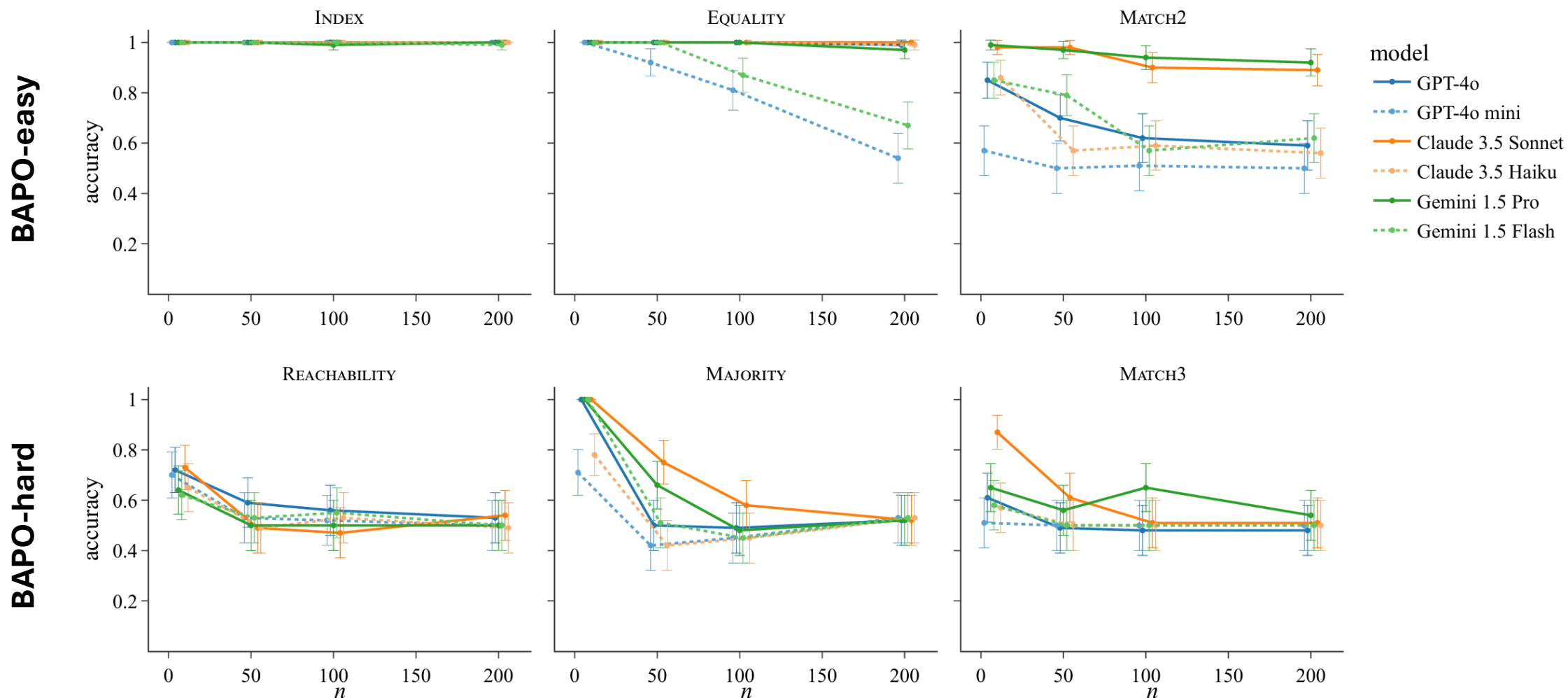
Some layer has $\sim \ell$ edges not in P

Generate $\ell! > 2^a$ prefixes by permuting them:

Given collision $f(x_i) = f(x_j)$, they
have some different s^*, t^* connectivity!

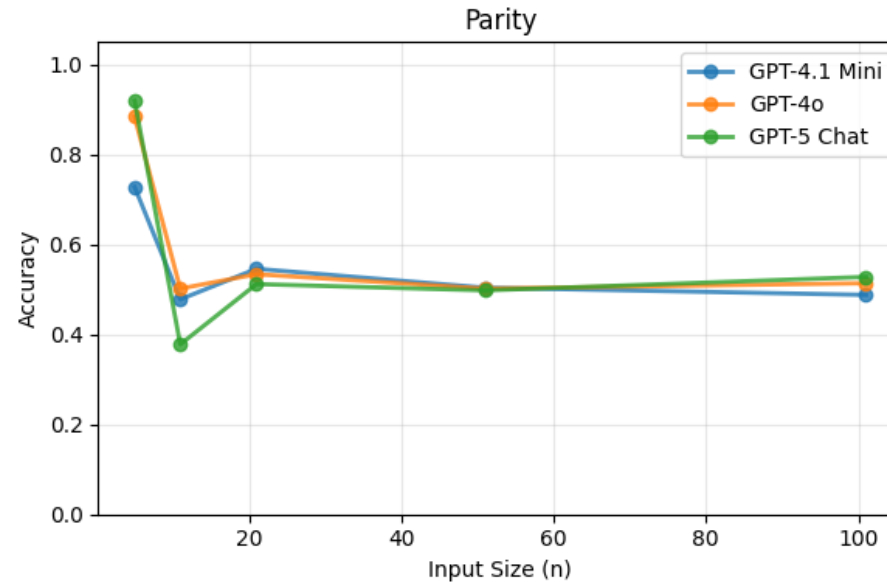


Revisiting the motivating problem



LLMs do fail on some BAPO-easy problems

Parity has $(1, 0)$ -BAPO, but:



Many reasons for failure, e.g., sensitivity [Hahn and Rofin 2024]

Conjecture. Without CoT/reasoning, frontier transformer-based LLMs will continue to fail on BAPO-hard problems beyond small n .

No counterexamples yet. Prove me wrong 😊

Talk outline

1. Background
2. The BAPO model [1]
3. BAPO results [1]
4. **BAPOs with chain of thought [2]**

[1] Schnabel, Tomlinson, Swaminathan, Neville. Lost in transmission: When and why LLMs fail to reason globally. *NeurIPS* 2025.

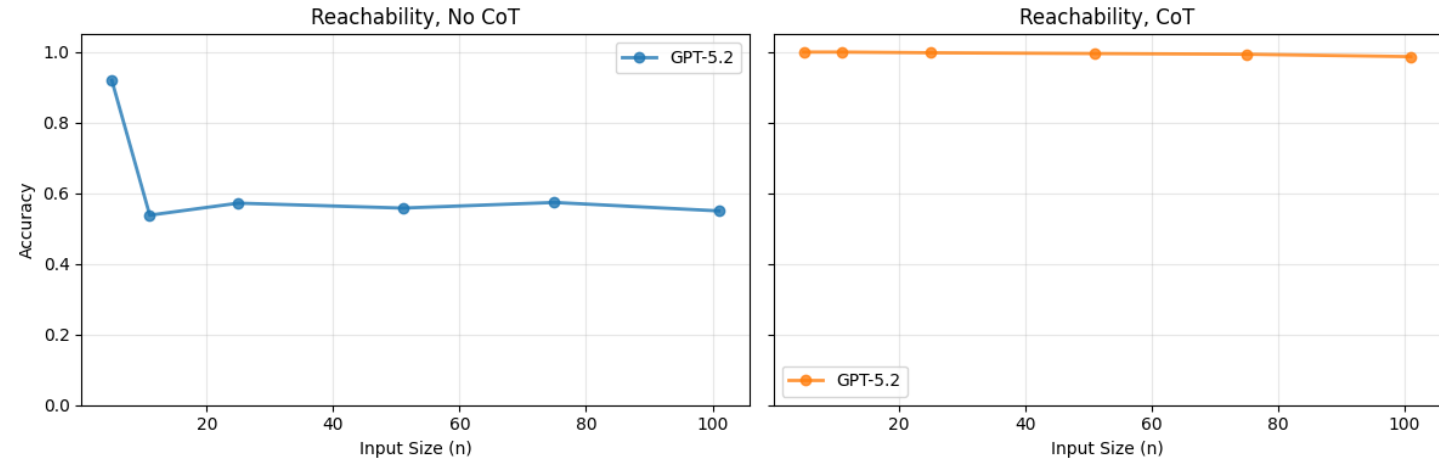
[2] Tomlinson, Schnabel, Swaminathan, Neville. Reasoning about reasoning: BAPO bounds on chain-of-thought token complexity in LLMs. *ICML* 2026.

[3] Altabaa, Schnabel, Tomlinson. Putting transformer theories to the test. *Working paper*, 2026.

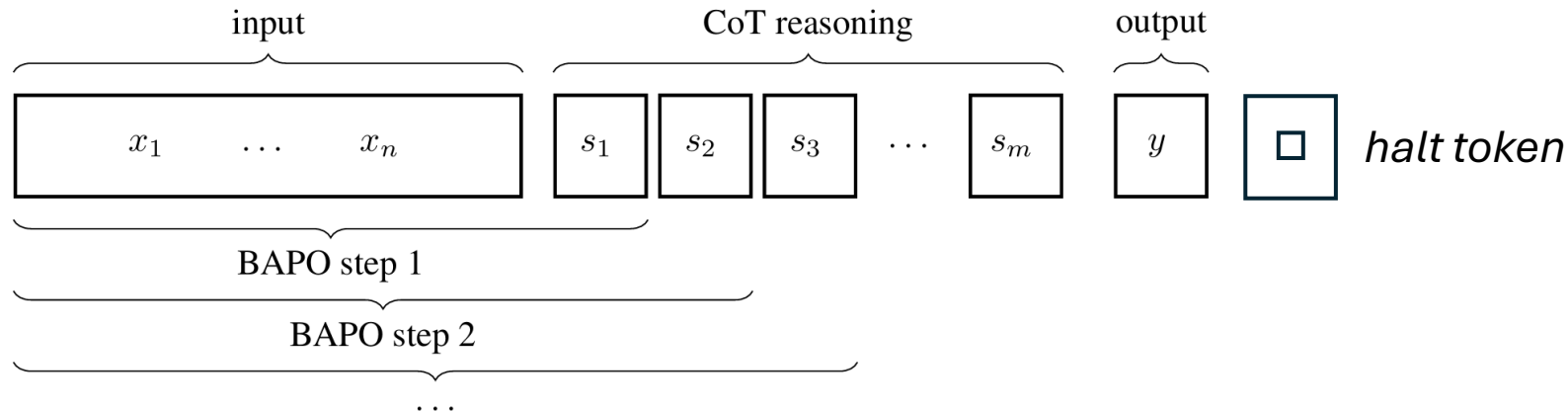
Modern LLMs don't just do one step, they reason

With CoT, transformers are Turing-complete [Pérez et al. 2021] [Merrill and Sabharwal 2024]

Empirically, it helps a lot!



Can define BAPO-CoT by applying a BAPO repeatedly:



Example: BAPO-CoT for Majority

Theorem. There is a $(0, 2)$ -BAPO-CoT that solves Majority using $O(n \log n)$ CoT steps.

[Tomlinson et al., 2026]

Idea: write counter in CoT, update one bit at a time

0111011101	000 001 010 ...
	
<i>input</i>	<i>CoT</i>

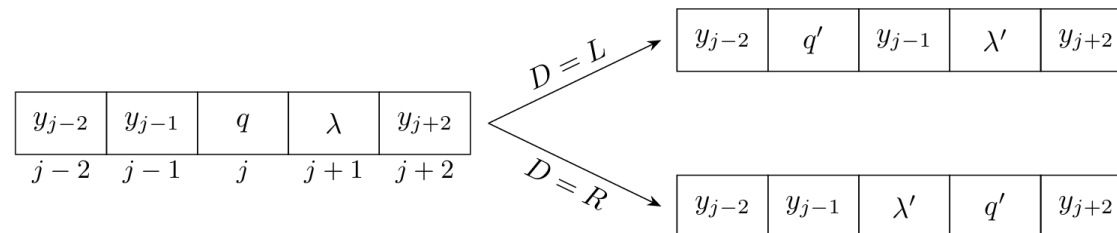
Each counter bit output only needs:
input bit being counted, previous counter bit*

Constant-bandwidth BAPO-CoTs can simulate Turing machines!

Theorem. Let L be decidable. For every n , there is a $(2, 3)$ -BAPO-CoT that solves $p(x) = 1[x \in L, |x| \leq n]$.

[Schnabel et al. 2025]

Idea: repeatedly write out tape in CoT, update one TM step at a time



Slow! Steps: $\Theta(t(n) \cdot \max(n, s(n)))$

Critical question for CoT: *how many steps are needed?*

Token complexity: time complexity for reasoning LLMs [Lee et al., 2025]

$\mathcal{P}_n$: size- n instances of a problem

$\mathcal{B}(\mathcal{P}_n)$: all constant-bandwidth BAPO-CoTs solving $\mathcal{P}_n$

Definition. The *BAPO token complexity* $c(n)$ for a problem n is:

$$c(n) = \min_{(f,g,h) \in \mathcal{B}(\mathcal{P}_n)} \max_{x \in \mathcal{P}_n} \text{cotTokens}((f, g, h), x)$$

Worst case over splits and attended sets G at every step

BAPO token complexity bounds

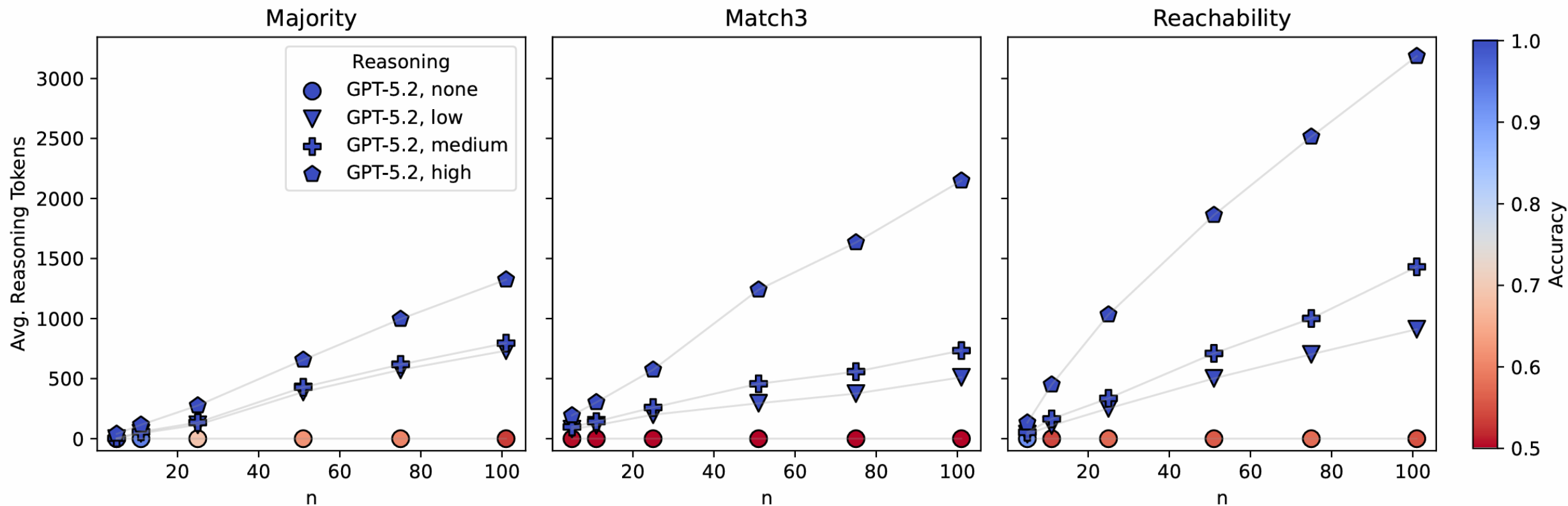
Corollary. Any problem decided by a TM in time $t(n)$ and space $s(n)$ has BAPO token complexity $O(t(n) \cdot \max(n, s(n)))$.

Theorem. Majority, Reachability, and Match3 have BAPO token complexity $\Omega(n)$.

[Tomlinson et al., 2026]

Proof idea: same as BAPO-hardness proof, but add prefix tokens to fool BAPO-CoT *at every CoT step*.

We see linear token scaling in practice!



A surprise

So far, BAPO-CoT seems well-behaved.

e.g., halting problem

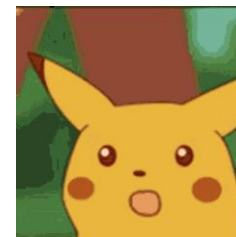
Theorem. For *any* decision problem and any n , there is a $(1, 1)$ -BAPO-CoT that solves it on size- n inputs in $\Theta(n)$ steps.

[Tomlinson et al., 2026]

Copy the input: $x_{1\dots n} \# x_{1\dots n}$

For every split, either:

1. first copy is entirely in prefix: $f(\text{prefix}) = x_{1\dots n} \in L$
2. second copy is entirely in suffix: $h(\text{suffix}) = x_{1\dots n} \in L$



+ our linear lower bounds are tight!

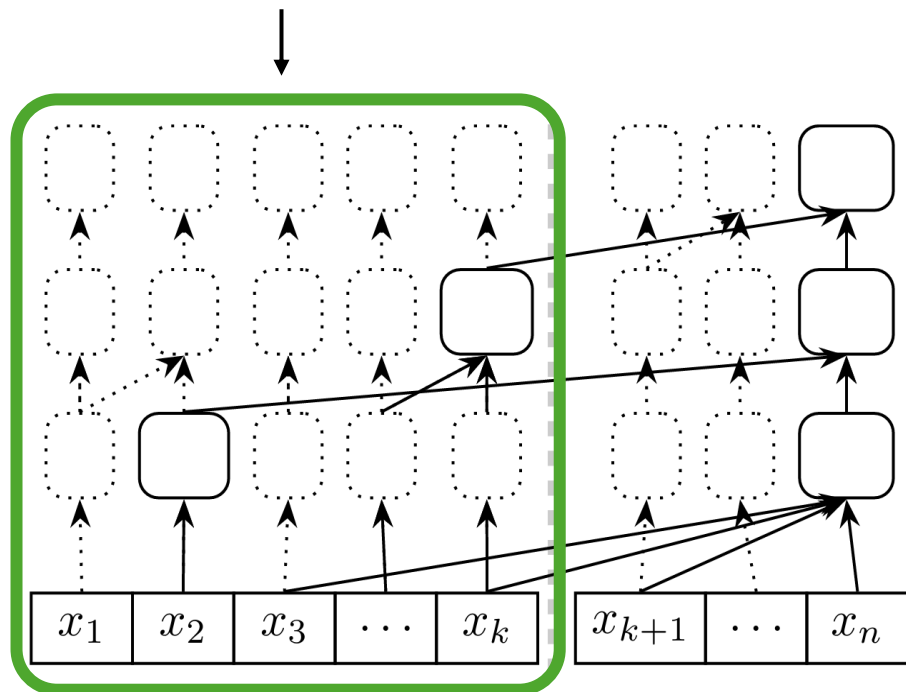
- this is not a useful model of reasoning LLMs...

Patching the input-doubling loophole

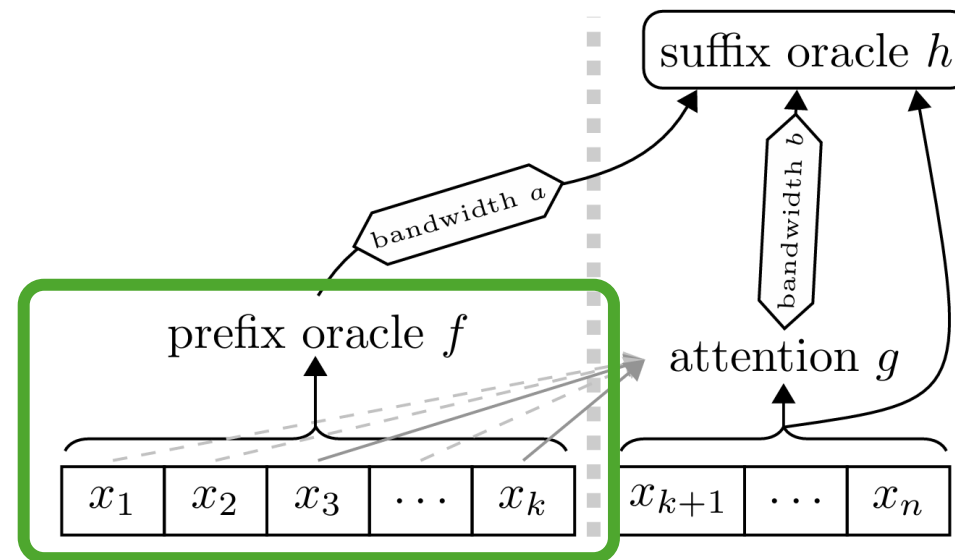
Without CoT, arbitrary split k prevented f and h from becoming too powerful (despite being any function)

$f(\text{prefix}) = x_{1\dots n} \in L$ $h(\text{suffix}) = x_{1\dots n} \in L \Rightarrow$ Need to weaken one of these

This is a transformer too!



f should only be able to compute BAPO-easy functions



Self-consistent BAPO

Definition. f is (a, b) -BAPO consistent for problem p if $\exists g, h$ s.t. (f, g, h) is an (a, b) -BAPO computing f :

$$f(x_{1\dots n}) = h(f(x_{1\dots k}), G, x_{k+1\dots n}, k)$$

(for all splits k and attended sets G)

Definition. An (a, b) -BAPO for which f is (a, b) -BAPO consistent is called *self-consistent*, denoted (a, b) -cBAPO.

Theorem. All results stated so far for BAPOs hold for cBAPOs, *except the input-doubling loophole!*

[Tomlinson et al., 2026]

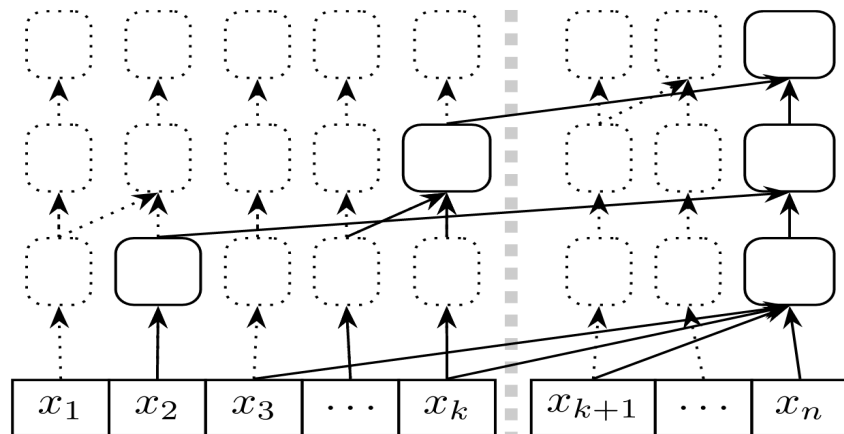
Concluding thoughts

1. Extending one-way communication with attention gives us a tractable model of constrained information flow in transformers
2. BAPO predictions align well with practice
3. This framework also lets us analyze CoT efficiency

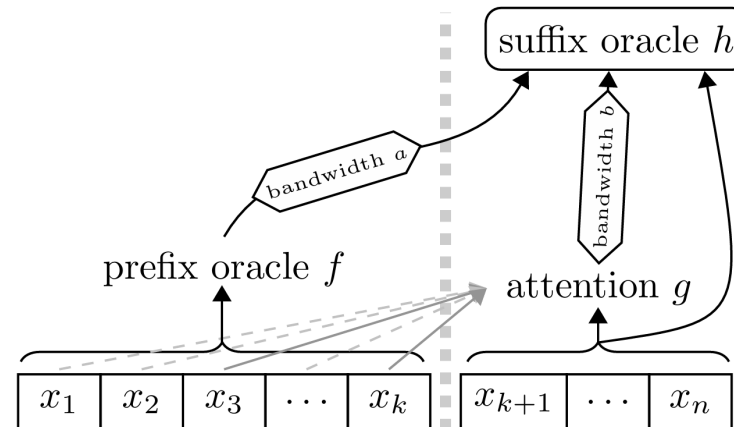
Ongoing / future work on BAPOs

1. Mechanistic interpretability: do BAPO information bottlenecks occur in real transformers?
2. Broader empirical exploration: is BAPO-hardness a good predictor of LLM / transformer failures?
3. A theory of agents:
single token LLMs $\rightarrow$ reasoning LLMs $\rightarrow$ agents with tools
BAPO BAPO-CoT ???
 - a) Complexity of determining the right tool calls / decomposition?
4. Open theoretical questions
 - a) Characterization of BAPO-easy problems
(have some for $(a, 0)$ - and $(0, b)$ -BAPOs)
 - b) Tight BAPO bandwidth bounds
 - c) cBAPO token complexity of problems (superlinear?)

Thank you! Questions?



BAPO bandwidth



BAPO token complexity

	Problem	Lower bound	Upper bound
BAPO-easy	INDEX (Thm. [1])		$(0, 1)$
	EQUALITY (Thm. [1])		$(1, 1)$
	DISJOINTNESS (Thm. [1])		$(1, 1)$
	MATCH 2_n (Thm. [5])		$(0, 1)$
	regular languages (Thm. [2])		$(\lceil \log_2 Q \rceil, 0)$
BAPO-hard	REACHABILITY (Thm. [3])	$(o(m^{1/c} \log m), o(m^{1-2/c}))$	trivial
	MAJORITY (Thm. [4])	$(o(\log n), o(n^{1-\epsilon}))$	$(\lceil \log_2 n \rceil, 0)$
	MATCH 3_n (Thm. [5])	$(o(n/b(n)), b(n))$	trivial
BAPO- Σ -hard	UNIQUE (Thm. [6])	$(o(\Sigma /b(\Sigma)), b(\Sigma))$	$(2 \Sigma , 0)$
	SETDIFF (Thm. [7])	$(o(\Sigma /b(\Sigma)), b(\Sigma))$	$(\Sigma , 0)$

Problem	Lower bounds	Upper bounds	
	cBAPO/BAPO	cBAPO	BAPO
MAJORITY	$\Omega(n)$ (Thm. 4.1)	$O(n \log n)$ (Thm. 5.6)	$n + 1$ (Prop. 5.4)
MATCH 3_n	$\Omega(n)$ (Thm. 4.2)	$O(n)$ (Thm. 5.7)	$n + 1$ (Prop. 5.4)
REACHABILITY	$\Omega(m)$ (Thm. 4.3)	$O(n^2)$ (Thm. 5.8)	$m + 3$ (Prop. 5.4)
Any decidable language		$O(t(n) \max(n, s(n)))^\dagger$ (Thm. 5.5)	$n + 1$ (Prop. 5.4)
Any language		(open)	$n + 1$ (Prop. 5.4)
Any cBAPO-easy problem		1 (by definition)	1 (by definition)

[1] Schnabel, Tomlinson, Swaminathan, Neville. Lost in transmission: When and why LLMs fail to reason globally. *NeurIPS* 2025.

[2] Tomlinson, Schnabel, Swaminathan, Neville. Reasoning about reasoning: BAPO bounds on chain-of-thought token complexity in LLMs. *ICML* 2026.

[3] Altabaa, Schnabel, Tomlinson. Putting transformer theories to the test. *Working paper*, 2026.